

Live Application and API Cost Guide

A practical offline guide to running validated tennis or basketball value signals, designing a BetBurger-style interface, and controlling monthly data costs.

Recommended starting point

Build a private live dashboard, but keep the actionable rule fixed at the validated decision time - currently around 60 minutes before the match. Poll every 60 to 120 seconds only near that window, notify the user, recheck the bookmaker price, and place manually. Budget about USD 30 per month for the first production pilot.

This guide answers

- How the live system should run from odds collection to settlement.
- Why a 30-minute call is suitable for discovery but not actionable pricing.
- How fixed T-60 deployment differs from continuous scanning.
- What the dashboard and alerts should show.
- How many API credits different polling schedules consume.
- What must be built before the current research repository is live-ready.

Scope: pre-match match-winner or moneyline research signals. This is decision-support software, not a guarantee of profit and not automatic bet placement.

1. How the live application should run

The best operating model is a small background service feeding a private dashboard. The mathematical signal calculation is fast; freshness, state management, execution checks, and auditability are the real operational work.

1

Collect and preserve

Fetch current prices and store the exact raw response with provider timestamps and quota headers.

2

Normalize and validate

Resolve fixtures and participants; reject incomplete, stale, suspended, conflicting, or impossible prices.

3

Calculate the probability

De-vig the reference market, apply the validated calibration artifact, and derive conservative p_{safe} .

4

Evaluate executable value

Use the currently offered price: $EV_{\text{safe}} = \text{executable odds} \times p_{\text{safe}} - 1$.

5

Create or update signal state

Track new, active, changed, expired, placed, rejected, and settled signals without duplicate alerts.

6

Display and notify

Show the audit trail in a dashboard and send a short phone alert only when a signal becomes actionable.

7

Record execution and settlement

Capture the price actually obtained, stake, rejection or limit, result, closing value, and bankroll exposure.

The 30-minute answer

A 30-minute schedule is acceptable for discovering fixtures or maintaining a watchlist. It is too slow for actionable odds because the current tennis configuration rejects quotes older than 180 seconds and short-lived value can disappear between checks.

2. Preserve the validated decision rule

A live deployment is only a valid continuation of the backtest when it makes decisions under the same timing rule. The present NBA and Wimbledon research designs evaluate entry around T-60. That should remain the first production rule.

Question	Fixed T-60 model	Continuous scanner
When can it bet?	One validated window near 60 minutes before start.	Whenever the first qualifying signal appears.
What can the UI show earlier?	A non-actionable watchlist.	Actionable signals throughout the scan period.
Backtest required	Point-in-time prices at the fixed decision time.	Complete timelines with first-trigger, expiry, cooldown, and duplicate rules.
Main risk	Missing a later improvement after the decision.	Strategy drift and repeated testing of the same match.
Recommendation now	Use this for initial live deployment.	Only after it is separately validated.

Recommended polling schedule

Purpose	Cadence	Meaning
Discover tournament keys	Every 30-60 minutes	Quota-free sports endpoint.
Discover fixtures	Every 30-60 minutes	Use the quota-free events endpoint where available.
More than 6 hours away	Every 15-30 minutes	Watchlist only; or do not request paid odds yet.
Approaching decision	Every 2-5 minutes	Prepare identity, coverage, and freshness checks.
T-75 to T-45	Every 60-120 seconds	Actionable pricing window for the fixed T-60 strategy.
After an alert	Immediate recheck	Confirm the offered price still meets EV_safe before placement.

Signal lifecycle

WATCHING -> NEW -> ACTIVE -> PLACED / REJECTED / EXPIRED -> SETTLED

Each state change should be idempotent: rerunning a job must never send the same alert or create the same intended bet twice. Any persistence filter, such as requiring two consecutive observations, must itself be included in historical validation before it becomes an entry rule.

Practical operating rule

Keep the dashboard continuously visible, but color rows as WATCHING until the validated decision window opens. This gives the familiar BetBurger experience without changing the strategy.

3. API credit and monthly cost model

The Odds API charges credits rather than tokens. The current project request uses one market (h2h) and two regions (uk and eu), so each successful call normally costs 2 credits. One tournament-level call returns all upcoming events under that sport key.

Published plans as of 8 August 2026

Plan	Monthly price	Credits	Best use here
Starter	Free	500	Very small shadow test.
20K	USD 30	20,000	Recommended fixed-time live pilot.
100K	USD 59	100,000	Fast polling across several active keys.
5M	USD 119	5,000,000	Large continuous scanner.
15M	USD 249	15,000,000	High-volume multi-sport operation.

Continuous polling estimate

Assumption: 12 active hours per day, 30 days, h2h, uk plus eu. Costs double if the service runs the same cadence for 24 hours.

Interval	One active key	Two active keys	Likely plan
30 minutes	1,440 credits	2,880 credits	USD 30
5 minutes	8,640 credits	17,280 credits	USD 30
2 minutes	21,600 credits	43,200 credits	USD 59
1 minute	43,200 credits	86,400 credits	USD 59

Formula

Monthly credits = active tournament keys x polls per day x active days x region equivalents x markets.

A cheaper fixed-window example

Eight separate decision windows per day, three calls around each window, two credits per call, and 30 days gives: $8 \times 3 \times 2 \times 30 = 1,440$ credits per month. Overlapping windows inside the same tournament key reduce the real total because one call covers every event.

Historical-download interaction

The existing NBA 2023-24 pilot can require up to 14,840 historical credits. If that download and live scanning occur in the same 20K billing month, only 5,160 credits remain. Keep the one-time historical download separate or temporarily use the 100K plan.

4. The BetBurger-style interface

The familiar part should be the fast, sortable signal table. The important difference is semantic: BetBurger often shows an arbitrage relationship, while this system shows an estimated and uncertainty-adjusted expected value. It must never imply guaranteed profit.

Essential dashboard columns

Group	Fields to show
Event	Sport, tournament, match, scheduled start, and countdown.
Offer	Selection, bookmaker, current executable odds, and direct link if supplied.
Probability	Raw market probability, calibrated probability, p_safe, and fair odds.
Value	Raw EV, EV_safe, minimum threshold, and distance above threshold.
Data quality	Quote age, reference-book age, bookmaker count, overround, dispersion, and flags.
Risk	Suggested maximum stake, available bankroll, open exposure, and concentration.
Lifecycle	Watching, new, active, changed, placed, rejected, expired, or settled.
Audit	Model version, config version, signal time, snapshot IDs, and calculation ID.

Alert behavior

- Send a phone notification only when a row first becomes actionable.
- Do not alert again unless price, EV_safe, status, or expiry changes materially.
- Include match, selection, bookmaker, odds, EV_safe, quote age, and expiry time.
- Require a fresh price check before the user confirms placement.
- Capture placed price, stake, rejection, limit, or changed odds from the same screen.

One-screen rule

The user should be able to answer four questions without opening another page: Is the signal still live? Why does the model consider it valuable? What can I safely stake? What evidence will be stored if I act?

Suggested row emphasis

Color	Meaning
Grey	Watchlist only; outside the validated decision window.
Blue	Actionable and all data-quality checks pass.
Amber	Manual review required: unusual edge, dispersion, or aging quote.
Red	Expired, stale, missing reference, suspended, or no longer executable.
Green	User recorded the bet as placed at a verified price.

5. Safe launch sequence

The first live objective is to validate execution, not to maximize turnover. A strong historical model can still fail live because the displayed price is stale, unavailable, limited, rejected, or already moving.

Phase	What happens	Evidence required to advance
1. Shadow	Signals and alerts run live; no money is placed.	Availability rate, quote decay, missing books, alert latency, and closing value.
2. Manual small stake	User rechecks and places manually; every failure is recorded.	Actual odds versus signalled odds, rejections, limits, slippage, and operational errors.
3. Controlled staking	Use p_{safe} and capped fractional Kelly with a real exposure ledger.	Stable execution, calibrated live outcomes, acceptable drawdown, and open-risk controls.
4. Automation	Consider placement integration only after extensive operational proof.	Duplicate protection, authentication, failure recovery, legal/provider compliance, and kill switch.

Minimum controls before real stakes

- Load a frozen, versioned calibration artifact; never refit it using future outcomes.
- Calculate EV from executable odds and conservative p_{safe} , not raw consensus alone.
- Reserve stake immediately for unresolved bets and prevent cash reuse across overlapping matches.
- Cap stake per bet, event, sport, bookmaker, day, and total open portfolio.
- Fail closed when the sharp reference is missing, stale, incomplete, or after the decision time.
- Persist raw provider evidence and the exact inputs used for every candidate.
- Provide a kill switch and health alarm for stalled collection, quota exhaustion, and clock errors.

Staking principle

Conservative sizing

Stake fraction = $\min(\text{position cap}, \text{Kelly fraction} \times \max(0, (\text{executable odds} \times p_{\text{safe}} - 1) / (\text{executable odds} - 1)))$.
During the first manual phase, small flat stakes are easier to audit and may be preferable until execution quality is proven.

What not to automate first

Do not begin with unattended placement, automatic approval of unknown player identities, or aggressive polling across every tournament. Those increase operational risk before the live data and execution pathway have been measured.

6. Current repository readiness

The repository is a strong research base but is not yet a live service. Tennis currently collects, stores, approves identities, normalizes, and evaluates through separate commands. Basketball contains chronological calibration and p_safe research logic, but its production live lifecycle is not implemented.

Already present	Still needed for live operation
Raw response preservation and point-in-time timestamps.	Long-running scheduler or worker with retries and health checks.
Typed normalization and conservative identity resolution.	Continuous normalization for known identities and quarantine queue for unknowns.
De-vigging, reference/consensus pricing, EV, and quality flags.	Versioned calibration artifact loaded by the live signal engine.
Freshness and minimum-bookmaker checks.	Persistent signal state, expiry, deduplication, and alert delivery.
SQLite provenance and historical backtest/reporting.	Dashboard API, authenticated interface, and deployment lifecycle.
Backtest staking calculations.	Executable-price verifier, open-exposure bankroll ledger, and settlement worker.

Recommended implementation order

Order	Deliverable	Completion test
1	Freeze live strategy contract	Exact sport, market, books, T-60 window, threshold, p_safe, expiry, and stake policy documented.
2	Background collection worker	Resumes safely, observes quotas, stores raw bytes, and raises health alarms.
3	Live signal state database	No duplicates; every change and expiry is reproducible from stored snapshots.
4	Read-only dashboard and alerts	Operator can explain, filter, and audit each row from one screen.
5	Shadow execution capture	Actual availability, changed price, rejection, and limit are recorded.
6	Exposure ledger and settlement	Overlapping stakes are reserved and released at actual settlement.
7	Controlled live pilot	Small manual stakes operate under caps and a kill switch.

Bottom-line recommendation

Start with The Odds API 20K plan at USD 30 per month. Use quota-free fixture discovery, adaptive polling, and 60-to-120-second refreshes only near the validated T-60 window. Run shadow mode first, then manual small stakes. Move to the USD 59 plan only if measured live execution shows that faster or broader polling adds value.

7. Travel cheat sheet

If you remember only five points

1) Live behavior must match the validated entry rule. 2) Thirty minutes is for discovery, not actionable odds. 3) Recheck every alert before placement. 4) Record actual execution, not just model output. 5) USD 30 per month is the sensible starting data budget.

Decision checklist

- Is the match inside the validated decision window?
- Are at least the required bookmakers present and fresh?
- Is the sharp/reference quote valid and timestamped before the decision?
- Is the target bookmaker price still available right now?
- Does EV_safe still exceed the locked threshold using the executable price?
- Are there quality flags requiring manual review?
- Does the stake fit bankroll, position, concentration, and open-exposure caps?
- Has this match or selection already been placed or rejected?

Quick cost reference

Operating style	Approximate monthly data budget
Small shadow test	Free to USD 30
Fixed T-60 manual alerts	USD 30
1-2 minute scan across several active keys	USD 59
Large continuous multi-sport scanner	USD 119 or a quoted WebSocket feed

Sources and verification

Prices, quotas, coverage, and provider behavior can change. Recheck the provider pages before subscribing.

The Odds API pricing and access: <https://the-odds-api.com/#get-access>

The Odds API v4 quota documentation: <https://the-odds-api.com/liveapi/guides/v4/>

The Odds API update intervals: <https://the-odds-api.com/sports-odds-data/update-intervals.html>

OddsPapi WebSocket production flow: <https://docs.oddsapi.io/websocket/overview>

OddsPapi public plan overview: <https://oddsapi.io/en>

Project source: `configs/research.toml`, `README.md`, `src/tennis_value`, and `src/basketball_value`.

Research warning

A positive backtest or live signal does not guarantee profit. Data latency, stale quotes, bookmaker limits, rejected bets, model error, and changing market conditions can remove the apparent edge. Preserve assumptions, uncertainty, and execution evidence.